Language Translation Tool

Project Report - Task 1

1. Objective

Build a web-based tool where a user types text, chooses a source and a target language, and receives the translated text on screen, using a translation API.

2. Requirements and how they are met

Task requirement	Implementation
User interface for text, source and target language	Text area, two language dropdowns (20 languages plus auto-detect), swap button
Use a translation API	MyMemory Translation API (free, no key required)
Send text to the API, get response	JavaScript fetch() GET request; JSON response parsed
Display translated text clearly	Dedicated output panel with loading and error messages
Optional: copy and text-to-speech	Copy button (Clipboard API) and Listen buttons (Web Speech API)

3. Technologies used

- HTML5 and CSS3 (responsive layout, light and dark theme)
- Vanilla JavaScript (no frameworks or build step)
- MyMemory Translation REST API
- Browser APIs: Clipboard API and SpeechSynthesis (text-to-speech)

4. How it works

- The user enters text (up to 500 characters) and picks the source and target languages.
- On clicking Translate, the app builds a request: `api.mymemory.translated.net/get?q=TEXT&langpair=en|hi`.
- The API returns JSON; the translated text is read from `responseData.translatedText`.
- The result is shown in the output panel. Errors (empty input, no internet, API failure) show a clear message.
- Copy places the result on the clipboard; Listen reads the text aloud in the selected language.

5. How to run

- Save the source code (section 7) as `translator.html`, or use the provided file.
- Open it in any modern browser (Chrome, Edge, Firefox) with an internet connection.
- Enter text, choose languages and click Translate (Ctrl+Enter also works).

6. Limitations and future scope

- The free MyMemory tier allows 500 characters per request and a daily usage limit.
- Text-to-speech depends on the voices installed in the user's browser or device.
- Can be upgraded to Google Translate or Microsoft Translator by changing the URL in the translate() function and adding an API key (kept on a backend server for security).
- Possible additions: translation history, voice input, document translation.

7. Source code (translator.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Language Translation Tool</title>
<style>
  :root{--bg:#eef3f4;--ink:#12292e;--muted:#5b7479;--card:#fff;--line:#cfdcde;--accent:#0b6e7f;--accent-
    ink:#fff}
  @media (prefers-color-scheme:dark){:root{--bg:#0e1b1e;--ink:#e6f1f2;--muted:#8fa9ad;--card:#15272b;--
    line:#27434a;--accent:#3fb4c7;--accent-ink:#06191d}}
  *{box-sizing:border-box}
  body{margin:0;background:var(--bg);color:var(--ink);font-family:"Segoe UI",system-ui,sans-serif;min-
    height:100vh;display:flex;justify-content:center;padding:28px 16px}
  main{width:100%;max-width:860px}
  h1{font-size:1.8rem;margin:0 0 4px;letter-spacing:-.02em}
  p.sub{margin:0 0 20px;color:var(--muted)}
  .bar{display:flex;gap:10px;align-items:center;margin-bottom:14px;flex-wrap:wrap}
  select{flex:1;min-width:140px;padding:10px 12px;font-size:1rem;border:1px solid var(--line);border-
    radius:8px;background:var(--card);color:var(--ink)}
  button{font:inherit;cursor:pointer;border:1px solid
    var(--line);background:var(--card);color:var(--ink);padding:10px 14px;border-radius:8px}
  button:hover{border-color:var(--accent)}
  button.primary{background:var(--accent);color:var(--accent-ink);border-color:var(--accent);font-
    weight:600;padding:12px 24px}
  button:disabled{opacity:.5;cursor:default}
  :focus-visible{outline:3px solid var(--accent);outline-offset:2px}
  .panes{display:grid;grid-template-columns:1fr 1fr;gap:14px}
  @media(max-width:640px){.panes{grid-template-columns:1fr}}
  .pane{background:var(--card);border:1px solid var(--line);border-
    radius:12px;padding:12px;display:flex;flex-direction:column;gap:8px}
  textarea,.out{width:100%;min-
    height:190px;border:0;resize:vertical;background:transparent;color:var(--ink);font:inherit;font-
    size:1.1rem;line-height:1.5;padding:4px}
  textarea:focus{outline:none}
  .out{white-space:pre-wrap;overflow:auto}
  .out.ph{color:var(--muted)}
  .tools{display:flex;gap:8px;justify-content:space-between;align-items:center;color:var(--muted);font-
    size:.85rem}
  .tools div{display:flex;gap:8px}
  .tools button{padding:6px 10px;font-size:.9rem}
  .go{margin-top:14px;display:flex;gap:12px;align-items:center}
  #status{color:var(--muted);font-size:.9rem}
  #status.err{color:#c0392b}
</style>
</head>
<body>
<main>
  <h1>Language Translation Tool</h1>
  <p class="sub">Type text, pick the languages, and translate.</p>

  <div class="bar">
    <select id="src" aria-label="Source language"></select>
    <button id="swap" title="Swap languages" aria-label="Swap languages"><-></button>
    <select id="tgt" aria-label="Target language"></select>
  </div>

  <div class="panes">
    <section class="pane">
      <textarea id="input" maxlength="500" placeholder="Enter text to translate" aria-label="Text to
    translate"></textarea>
      <div class="tools"><span id="count">0 / 500</span>
        <div><button id="speakIn"> Listen</button><button id="clear">Clear</button></div></div>
    </section>
    <section class="pane">
      <div id="output" class="out ph" aria-live="polite">Translation appears here</div>
      <div class="tools"><span></span>
        <div><button id="speakOut" disabled> Listen</button><button id="copy"
    disabled>Copy</button></div></div>
    </section>
  </div>

  <div class="go"><button id="translate" class="primary">Translate</button><span id="status"></span></div>
</main>

<script>
const LANGS = {auto:"Detect language",en:"English",hi:"Hindi",es:"Spanish",fr:"French",de:"German",it:"Itali
    an",pt:"Portuguese",ru:"Russian",ja:"Japanese",ko:"Korean","zh-CN":"Chinese (Simplified)",ar:"Arabic",bn
    :"Bengali",ur:"Urdu",ta:"Tamil",te:"Telugu",mr:"Marathi",gu:"Gujarati",tr:"Turkish",nl:"Dutch"};
const $ = id => document.getElementById(id);
const src=$("src"), tgt=$("tgt"), input=$("input"), output=$("output"), status=$("status");
```

```
for (const [code,name] of Object.entries(LANGS)) {
  src.add(new Option(name, code));
  if (code!=="auto") tgt.add(new Option(name, code));
}
src.value="en"; tgt.value="hi";

input.addEventListener("input", () => $("count").textContent = input.value.length + " / 500");
input.addEventListener("keydown", e => { if ((e.ctrlKey||e.metaKey) && e.key==="Enter") translate(); });

function setStatus(msg, isErr=false){ status.textContent = msg; status.className = isErr ? "err" : ""; }
function setOutput(text){
  output.textContent = text || "Translation appears here";
  output.classList.toggle("ph", !text);
  $("copy").disabled = $("speakOut").disabled = !text;
}

async function translate(){
  const text = input.value.trim();
  if (!text) { setStatus("Enter some text first.", true); return; }
  if (src.value === tgt.value) { setOutput(text); setStatus("Source and target are the same."); return; }
  const btn = $("translate");
  btn.disabled = true; setStatus("Translating...");
  try {
    // MyMemory free translation API (no key needed). "Autodetect" is its code for auto source.
    const pair = (src.value==="auto" ? "Autodetect" : src.value) + "|" + tgt.value;
    const url = "https://api.mymemory.translated.net/get?q=" + encodeURIComponent(text) + "&langpair=" +
    encodeURIComponent(pair);
    const res = await fetch(url);
    if (!res.ok) throw new Error("Server returned " + res.status);
    const data = await res.json();
    if (data.responseStatus !== 200) throw new Error(data.responseDetails || "Translation failed");
    setOutput(data.responseData.translatedText);
    setStatus("Done.");
  } catch (err) {
    setOutput("");
    setStatus("Could not translate: " + err.message + ". Check your internet connection.", true);
  } finally { btn.disabled = false; }
}

function speak(text, lang){
  if (!("speechSynthesis" in window)) { setStatus("Text-to-speech is not supported in this browser.", true);
     return; }
  speechSynthesis.cancel();
  const u = new SpeechSynthesisUtterance(text);
  if (lang && lang !== "auto") u.lang = lang;
  speechSynthesis.speak(u);
}

$("translate").onclick = translate;
$("swap").onclick = () => {
  if (src.value==="auto") return setStatus("Choose a source language to swap.", true);
  [src.value, tgt.value] = [tgt.value, src.value];
  if (!output.classList.contains("ph")) { input.value = output.textContent; input.dispatchEvent(new
    Event("input")); setOutput(""); }
};
$("clear").onclick = () => { input.value=""; input.dispatchEvent(new Event("input")); setOutput("");
    setStatus(""); input.focus(); };
$("copy").onclick = async () => {
  try { await navigator.clipboard.writeText(output.textContent); setStatus("Copied to clipboard."); }
  catch { setStatus("Copy failed. Select the text and copy manually.", true); }
};
$("speakOut").onclick = () => speak(output.textContent, tgt.value);
$("speakIn").onclick = () => input.value.trim() && speak(input.value, src.value);
</script>
</body>
</html>
```